

L9a: Linear Models for Classification

Binary classification, the Perceptron, and the confusion matrix

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can a linear model assign each example to one of two classes, and how do we judge the labels it assigns?

Today's concepts

- Adapt a linear model for binary classification: pass its output through the sign function to get a class label, and see how the parameters define a boundary between the two classes.
- Train a linear classifier with the Perceptron algorithm: update the parameters only on mistakes, show why each update helps, and see when the algorithm is guaranteed to separate the classes.
- Evaluate a classifier with a confusion matrix: compute accuracy, precision, recall, and specificity, and see why accuracy alone can mislead when one class is rare.

Lecture: Linear Models for Classification

Lecture and Example

Lecture: Linear Models for Classification
In this lecture, we turn a linear model into a binary classifier, train it with the Perceptron algorithm, and evaluate its predictions with a confusion matrix.

Example: Build a Perceptron Classifier
In this example, we train the Perceptron to tell genuine banknotes from forged ones, classify banknotes it has not seen, and use a confusion matrix to see which kinds of errors it makes.

Binary Classification Problem

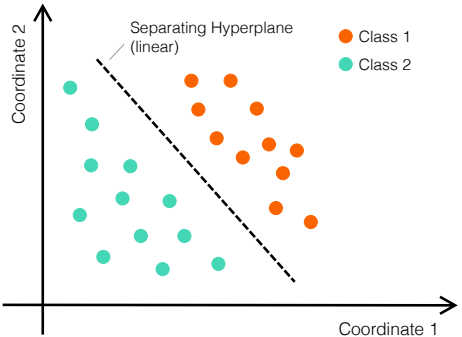
In a binary classification problem, each example belongs to one of two classes, and we want to predict the class from the example's features. A linear model computes a continuous output, a score, from the features. An activation function σ transforms the score either directly into a class label or into the probability of each class.

- The Perceptron (Rosenblatt, 1957) learns a linear decision boundary and maps the score directly to a class label in $\{-1, +1\}$ with $\sigma(z) = \texttt{sign}(z)$.
- Logistic regression uses the logistic function, an S-shaped curve with values between 0 and 1, to turn the score into the probability that an example belongs to a class.

Today, we focus on the Perceptron.

Linearly Separable Data

Each point is an example with two features, colored by its class.



These data are linearly separable: a straight boundary puts every class 1 example on one side and every class 2 example on the other.

The Perceptron Classifier

Suppose a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid i = 1, \ldots, n\}$ has n labeled examples, with features $\mathbf{x}_i \in \mathbb{R}^m$ and a class label $y_i \in \{-1, 1\}$. The Perceptron estimates the label by passing the linear score through the sign function:

$$\hat{y}_i = \texttt{sign}(\hat{\mathbf{x}}_i^\top \theta), \qquad \texttt{sign}(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ -1 & \text{if } z < 0, \end{cases}$$

where the augmented features $\hat{\mathbf{x}}_i^\top = (x_{i1}, \ldots, x_{im}, 1)$ add a 1 for the bias term, so $p = m + 1$, and $\theta = (w_1, \ldots, w_m, b) \in \mathbb{R}^p$ holds the weight w_j of each feature and the bias b.

Decision Boundary and Mistakes

The decision boundary is the set of feature vectors where the score is zero:

$$\{\mathbf{x} \in \mathbb{R}^m \mid \hat{\mathbf{x}}^\top \theta = w_1 x_1 + \cdots + w_m x_m + b = 0\}.$$

The score is positive on one side and negative on the other. When at least one weight is nonzero, the boundary is a hyperplane: a line when $m = 2$ and a plane when $m = 3$. The weights set its orientation, and the bias shifts it away from the origin.

The product $y_i \, \hat{\mathbf{x}}_i^\top \theta$ is positive when example i is on the correct side, and example i is a mistake when $y_i \, \hat{\mathbf{x}}_i^\top \theta \leq 0$: on the wrong side or on the boundary. The dataset is linearly separable if some θ makes this product positive for every example.

Online Perceptron Training

Initialize: Given $\mathcal{D}$, a pass limit $T \geq 1$, and a mistake threshold M (e.g., $M = 0$), set $\theta = (w_1, \ldots, w_m, b)$ to zero or small random values and the loop counter $t \leftarrow 0$.

While `true` **do**:

1. Initialize the number of mistakes `mistakes` $\leftarrow 0$.
2. For each training example $(\mathbf{x}, y) \in \mathcal{D}$, with augmented features $\hat{\mathbf{x}}$, **do**:
   - If $y\,\hat{\mathbf{x}}^\top \theta \leq 0$, **then** the example is a mistake. Update $\theta \leftarrow \theta + y\,\hat{\mathbf{x}}$ and increment `mistakes` $\leftarrow$ `mistakes` $+ 1$.
3. After processing all examples, increment $t \leftarrow t + 1$.
4. If `mistakes` $\leq M$ **or** $t \geq T$, **then** return θ. Otherwise, repeat from step 1.

The Perceptron is an online algorithm: it looks at one example at a time and updates θ right after each mistake.

Why Does the Update Help?

Suppose the example $(\mathbf{x}, y)$ is a mistake, and let $\theta' = \theta + y\,\hat{\mathbf{x}}$ be the updated parameters. Because $y^2 = 1$:

$$y\,\hat{\mathbf{x}}^\top \theta' = y\,\hat{\mathbf{x}}^\top \theta + y^2\,\hat{\mathbf{x}}^\top \hat{\mathbf{x}} = y\,\hat{\mathbf{x}}^\top \theta + \|\hat{\mathbf{x}}\|_2^2.$$

- The product grows by $\|\hat{\mathbf{x}}\|_2^2$, the squared length of $\hat{\mathbf{x}}$, which is at least 1 because the last entry of $\hat{\mathbf{x}}$ is 1. The update moves this example's score toward the correct sign.
- One update may not be enough: the product may still be negative, and the update also changes the scores of the other examples. So we pass through the data repeatedly.

Perceptron Convergence Theorem

Novikoff (1962). Suppose that:

- Margin: some $\theta^\star$ with $\|\theta^\star\|_2 = 1$ gives $y_i\, \hat{\mathbf{x}}_i^\top \theta^\star \geq \gamma > 0$ for every example.
- Radius: $R = \max_i \|\hat{\mathbf{x}}_i\|_2$ is the length of the longest augmented feature vector.

Then, starting from $\theta = \mathbf{0}$, the number of updates k satisfies:

$$\boxed{k \leq \left(\frac{R}{\gamma}\right)^2}$$

A larger margin γ lowers the bound. On separable data, with $M = 0$ and a large enough T, the Perceptron stops after finitely many updates.

When Does the Perceptron Stop?

- Separable data: a pass with zero mistakes leaves θ unchanged, so the returned θ classifies every training example correctly.
- Data that are not separable: every pass makes at least one mistake, so with $M = 0$ the loop runs until it reaches the limit of T passes.
- In practice: we rarely know in advance whether the data are separable. That is why the algorithm always has a pass limit T, and why we may stop after a pass with at most $M > 0$ mistakes.

Another way to train a linear classifier is to write a loss function that measures the classification error and minimize it with numerical optimization.

Confusion Matrix for Binary Classification

A confusion matrix compares predicted labels with actual labels. We call the class we want to detect positive ($y = 1$) and the other class negative ($y = -1$):

	Predicted positive	**Predicted negative**
Actual positive	True positive (TP)	False negative (FN)
Actual negative	False positive (FP)	True negative (TN)

The diagonal entries count the correct predictions, and the off-diagonal entries count the two kinds of mistakes.

In medical diagnosis, a false positive tells a healthy patient they have the disease, while a false negative misses a patient who has it. The two mistakes can have very different costs, so we count them separately.

Performance Metrics

The confusion matrix gives the counts for four performance metrics:

Metric	Formula	What it measures
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Fraction of all predictions that are correct
Precision	$\frac{TP}{TP+FP}$	Fraction of positive predictions that are correct
Recall (sensitivity)	$\frac{TP}{TP+FN}$	Fraction of actual positives the classifier finds
Specificity	$\frac{TN}{TN+FP}$	Fraction of actual negatives labeled negative

Accuracy can mislead when one class is rare: if 3% of examples are positive, always predicting negative is 97% accurate, with zero recall.

Example: Build a Perceptron Classifier

In this example, we classify 1,372 genuine and forged banknotes from four numerical features computed from a camera image of each note.

1. Train the Perceptron on the training banknotes, which are not linearly separable, and keep the best parameters with the pocket algorithm.
2. Classify the testing banknotes, which the Perceptron never saw, and compare the testing and training mistake rates.
3. Compute the confusion matrix for the testing banknotes, with forged banknotes as the positive class, and the four performance metrics.

A false negative accepts a forgery as genuine; a false positive flags a genuine banknote as forged.

Example: Build a Perceptron classifier

Lab: The Perceptron on XOR

In the lab, we train the Perceptron on datasets that are and are not linearly separable, including the XOR (exclusive or) pattern:

- the feature plane is cut into four wedges, and diagonally opposite wedges share a class,
- no straight line separates the classes, so every pass makes a mistake,
- we compare the learned boundaries and the test performance, with and without the pocket algorithm.

Minsky and Papert's 1969 proof that the Perceptron cannot learn XOR became a central point of their critique.

Lab: The Perceptron on non-linearly separable data

Summary

We adapted linear models for binary classification, trained a linear classifier with the Perceptron, and evaluated it with a confusion matrix.

Key takeaways

- **Linear models for classification.** We passed the linear score through the sign function to get a class label; the parameters define the decision boundary between the two classes.
- **Perceptron learning and convergence.** We showed that each update moves a mistake's score toward the correct sign, so on separable data the Perceptron stops after finitely many updates.
- **Performance evaluation.** We used the confusion matrix counts to compute four metrics and saw that accuracy alone can hide a classifier that misses a rare class.

The example and lab apply these ideas to banknotes and to the XOR pattern.